

# Lesson 7 Notes: Lower Triangular Matrix in C

## 1. Main idea of Lesson 7

Lesson 7 teaches how to implement a **lower triangular matrix in C**.

In Lessons 5 and 6, we mainly focused on formulas:

- Lesson 5: lower triangular matrix using **row-major mapping**
- Lesson 6: lower triangular matrix using **column-major mapping**

In Lesson 7, we now turn those formulas into a real **C program**.

The main tools used in this lesson are:

```
struct Matrix
```

and:

```
malloc()
```

So Lesson 7 is about combining:

- lower triangular matrix rules
- 1D array storage
- C structs
- dynamic memory allocation
- `set`, `get`, and `display` functions
- row-major formula first
- column-major formula as an update
- the difference between `m->n` and `m.n`

---

## 2. What you should already know before Lesson 7

Before Lesson 7, you should understand these ideas:

| Concept         | Meaning                                                                  |
|-----------------|--------------------------------------------------------------------------|
| Special matrix  | A matrix with a pattern that helps us save memory.                       |
| Structural zero | A zero that exists because of the matrix rule, not because we stored it. |

| Concept                 | Meaning                                                               |
|-------------------------|-----------------------------------------------------------------------|
| 1D array storage        | Storing useful matrix values in one array instead of a full 2D array. |
| Diagonal matrix         | Stores values only where <code>i == j</code> .                        |
| Lower triangular matrix | Stores values where <code>i &gt;= j</code> .                          |
| Row-major storage       | Stores values row by row.                                             |
| Column-major storage    | Stores values column by column.                                       |
| C struct                | Groups related data together.                                         |
| Pointer                 | Stores the address of another value or memory block.                  |
| Dynamic memory          | Memory created while the program is running.                          |
| <code>malloc()</code>   | Allocates memory in C.                                                |
| <code>free()</code>     | Releases memory allocated by <code>malloc()</code> .                  |

The most important idea from Lesson 5 is the lower triangular row-major formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

The most important idea from Lesson 6 is the lower triangular column-major formula:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

Lesson 7 uses these formulas inside a C program.

### 3. Quick progress review

So far, the project has moved like this:

#### Lessons 1 to 4: Diagonal matrix

A diagonal matrix stores only the main diagonal.

Condition:

```
i == j
```

Index formula:

```
index = i - 1
```

Example:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

Stored array:

```
A = [5, 8, 9, 12]
```

### Lesson 5: Lower triangular row-major mapping

A lower triangular matrix stores the main diagonal and everything below it.

Condition:

```
i >= j
```

Row-major formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

### Lesson 6: Lower triangular column-major mapping

The matrix shape is still lower triangular.

Condition still stays:

```
i >= j
```

But the storage order changes from row by row to column by column.

Column-major formula:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

## Lesson 7: Lower triangular matrix in C

Now we implement the lower triangular matrix using:

```
struct Matrix
```

and:

```
malloc()
```

This lesson turns the formula into a working C program.

---

## 4. What is a lower triangular matrix?

A **lower triangular matrix** is a square matrix where all useful values are on or below the main diagonal.

Example:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

The useful part is:

```
1
2 3
4 5 6
7 8 9 10
```

The values above the main diagonal are zeros.

These zeros are **structural zeros**.

They are not stored in memory.

---

## 5. Stored condition

For a lower triangular matrix, a position is stored when:

```
i >= j
```

Where:

```
i = row number  
j = column number
```

Examples:

| Position | Check  | Stored? |
|----------|--------|---------|
| M[1,1]   | 1 >= 1 | Yes     |
| M[2,1]   | 2 >= 1 | Yes     |
| M[2,2]   | 2 >= 2 | Yes     |
| M[3,1]   | 3 >= 1 | Yes     |
| M[4,3]   | 4 >= 3 | Yes     |
| M[4,4]   | 4 >= 4 | Yes     |

So the main rule is:

```
if (i >= j)  
    store or read from the array;
```

## 6. Zero condition

For a lower triangular matrix, a position is zero when:

```
i < j
```

That means the position is above the main diagonal.

Examples:

| Position | Check | Meaning         |
|----------|-------|-----------------|
| M[1,2]   | 1 < 2 | Structural zero |

| Position | Check | Meaning         |
|----------|-------|-----------------|
| M[1,3]   | 1 < 3 | Structural zero |
| M[2,4]   | 2 < 4 | Structural zero |
| M[3,4]   | 3 < 4 | Structural zero |

These values are not stored.

The program simply returns or prints 0 when needed.

## 7. Stored region vs zero region

For this matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9 10
```

Stored positions are:

```
M[1,1]
M[2,1] M[2,2]
M[3,1] M[3,2] M[3,3]
M[4,1] M[4,2] M[4,3] M[4,4]
```

Zero positions are:

```
M[1,2] M[1,3] M[1,4]
M[2,3] M[2,4]
M[3,4]
```

Stored condition:

```
i >= j
```

Zero condition:

$$i < j$$

## 8. Why lower triangular storage saves memory

A normal  $N \times N$  matrix stores:

$N^2$  values

A lower triangular matrix stores only:

$N(N + 1) / 2$  values

This is about half of the full matrix.

Example for  $N = 4$ :

```
Full matrix values = 4 x 4 = 16
Lower triangular stored values = 4(4 + 1) / 2
                                = 4(5) / 2
                                = 20 / 2
                                = 10
```

So for a  $4 \times 4$  lower triangular matrix, we store only 10 values instead of 16.

Example for  $N = 5$ :

```
Full matrix values = 5 x 5 = 25
Lower triangular stored values = 5(5 + 1) / 2
                                = 5(6) / 2
                                = 30 / 2
                                = 15
```

So for a  $5 \times 5$  lower triangular matrix, we store only 15 values instead of 25.

## 9. Number of structural zeros

The number of structural zeros above the main diagonal is:

$$N(N - 1) / 2$$

For `N = 4`:

$$\begin{aligned}\text{Zeros} &= 4(4 - 1) / 2 \\ &= 4(3) / 2 \\ &= 12 / 2 \\ &= 6\end{aligned}$$

For `N = 5`:

$$\begin{aligned}\text{Zeros} &= 5(5 - 1) / 2 \\ &= 5(4) / 2 \\ &= 20 / 2 \\ &= 10\end{aligned}$$

So a lower triangular matrix has:

$$\begin{aligned}\text{stored values} &= N(N + 1) / 2 \\ \text{zero values} &= N(N - 1) / 2\end{aligned}$$

---

## 10. Why we use a C struct

In C, a `struct` is used to group related data together.

For a lower triangular matrix, we need to store two things:

1. The 1D array that stores the useful values
2. The matrix dimension `n`

So we create this struct:

```
struct Matrix {  
    int *A;  
    int n;  
};
```

This means the matrix has:

```
int *A;
```

and:

```
int n;
```

---

## 11. Meaning of `int *A`

```
int *A;
```

means `A` is a pointer to an integer.

In this lesson, `A` will point to a dynamic array.

That dynamic array stores only the useful lower triangular values.

Example:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

This array can represent the full matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

The zeros are not stored.

They are created logically when needed.

---

## 12. Meaning of `int n`

```
int n;
```

stores the dimension of the square matrix.

If:

```
m.n = 4;
```

then the matrix behaves like a:

4 x 4 matrix

Even though the visible matrix has:

4 x 4 = 16 positions

we only store:

$4(4 + 1) / 2 = 10$  values

So `n` tells the program how many rows and columns the matrix has.

---

## 13. Why we use dynamic memory

In earlier beginner examples, we may use a fixed array like this:

```
int A[10];
```

But that is not flexible.

If the user enters `n = 4`, we need `10` values.

If the user enters `n = 5`, we need `15` values.

If the user enters `n = 10`, we need:

$10(10 + 1) / 2 = 55$  values

So the array size depends on the user input.

That is why we use dynamic memory.

Dynamic memory lets us create the correct amount of memory while the program is running.

---

## 14. Header files needed

The program uses two header files:

```
#include <stdio.h>
#include <stdlib.h>
```

### Why `stdio.h` is needed

`stdio.h` is needed for:

```
printf()
scanf()
```

These functions are used for input and output.

### Why `stdlib.h` is needed

`stdlib.h` is needed for:

```
malloc()
free()
```

These functions are used for dynamic memory.

---

## 15. Allocating memory with `malloc`

The number of stored values is:

```
m.n * (m.n + 1) / 2
```

So we first calculate:

```
int totalElements = m.n * (m.n + 1) / 2;
```

Then we allocate memory:

```
m.A = (int *)malloc(totalElements * sizeof(int));
```

Full allocation code:

```
int totalElements = m.n * (m.n + 1) / 2;  
m.A = (int *)malloc(totalElements * sizeof(int));
```

## Meaning of this line

```
m.A = (int *)malloc(totalElements * sizeof(int));
```

This means:

1. `malloc()` asks for memory from the heap.
2. `totalElements * sizeof(int)` tells how many bytes are needed.
3. `(int *)` converts the returned memory address into an integer pointer.
4. `m.A` stores the address of that memory.

---

## 16. Why we multiply by `sizeof(int)`

`malloc()` allocates memory in bytes.

But we need memory for integers.

So we write:

```
totalElements * sizeof(int)
```

If `totalElements = 10`, then this means:

```
allocate enough bytes for 10 integers
```

This is better than assuming an integer always has a fixed byte size.

On many modern systems, an `int` is often 4 bytes, but the program should not depend on guessing that.

So using `sizeof(int)` is the correct C style.

---

## 17. Releasing memory with `free`

When we use `malloc()`, we must release the memory using `free()`.

At the end of the program, we write:

```
free(m.A);
```

This gives the memory back.

### Important rule

```
malloc() must be matched with free()
```

Correct:

```
m.A = (int *)malloc(totalElements * sizeof(int));  
free(m.A);
```

If we forget `free(m.A)`, the program may cause a memory leak.

A memory leak means the program asks for memory but does not return it.

---

## 18. Row-major storage in Lesson 7

Lesson 7 first implements the matrix using **row-major storage**.

Row-major means we store values row by row.

Example matrix:

```
1  0  0  0  
2  3  0  0  
4  5  6  0  
7  8  9  10
```

Row-major storage means:

Row 1: 1  
Row 2: 2 3  
Row 3: 4 5 6  
Row 4: 7 8 9 10

Stored array:

A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Mapping:

| Matrix position | Array position |
|-----------------|----------------|
| M[1,1]          | A[0]           |
| M[2,1]          | A[1]           |
| M[2,2]          | A[2]           |
| M[3,1]          | A[3]           |
| M[3,2]          | A[4]           |
| M[3,3]          | A[5]           |
| M[4,1]          | A[6]           |
| M[4,2]          | A[7]           |
| M[4,3]          | A[8]           |
| M[4,4]          | A[9]           |

## 19. Row-major index formula

For lower triangular row-major storage, the formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Use this formula only when:

$$i \geq j$$

If:

$$i < j$$

then the position is above the diagonal, so it is not stored.

---

## 20. Meaning of the row-major formula

The formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

It has two parts.

### Part 1: Skip previous rows

$$i * (i - 1) / 2$$

This counts how many stored values come before row  $i$ .

For example, if  $i = 4$ , previous rows are:

Row 1 stores 1 value  
Row 2 stores 2 values  
Row 3 stores 3 values

Total values before row 4:

$$1 + 2 + 3 = 6$$

Formula:

$$4 * (4 - 1) / 2 = 4 * 3 / 2 = 6$$

### Part 2: Move inside the current row

$$j - 1$$

This moves to the correct column inside the current row.

Examples:

| Column | Movement |
|--------|----------|
| j = 1  | 0 steps  |
| j = 2  | 1 step   |
| j = 3  | 2 steps  |
| j = 4  | 3 steps  |

So:

j - 1

converts the 1-based column number into a 0-based movement inside the row.

### Formula in plain English

index = number of stored values before this row + movement inside this row

## 21. The `set` function

The `set` function stores a value into the matrix.

Code:

```
void set(struct Matrix *m, int i, int j, int x) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        m->A[index] = x;
    }
}
```

### Meaning of parameters

```
struct Matrix *m
```

This is a pointer to the matrix.

```
int i
```

This is the row number.

```
int j
```

This is the column number.

```
int x
```

This is the value to store.

### What the function does

The function checks:

```
if (i >= j)
```

If true, the position belongs to the lower triangle.

Then it calculates the index:

```
int index = (i * (i - 1) / 2) + (j - 1);
```

Then it stores the value:

```
m->A[index] = x;
```

If `i < j`, the function does nothing.

Why?

Because positions above the main diagonal must stay zero.

---

## 22. Why `set` receives a pointer

The `set` function must modify the real matrix.

So it receives:

```
struct Matrix *m
```

This means `m` is a pointer to a `struct Matrix`.

When calling `set`, we pass the address of the matrix:

```
set(&m, i, j, x);
```

The `&m` means:

```
address of m
```

Because `set` receives the address, it can change the original matrix.

---

## 23. Why `set` uses `m->A`

Inside `set`, `m` is a pointer.

So we use the arrow operator:

```
m->A
```

not the dot operator:

```
m.A
```

Correct:

```
m->A[index] = x;
```

Wrong:

```
m.A[index] = x;
```

Simple rule:

If it is a pointer, use ->  
If it is a normal struct variable, use .

## 24. Dry run of `set` for a stored position

Call:

```
set(&m, 4, 3, 9);
```

Values:

```
i = 4  
j = 3  
x = 9
```

Step 1: Check condition.

```
4 >= 3
```

This is true.

So the value can be stored.

Step 2: Calculate index.

```
index = i * (i - 1) / 2 + (j - 1)  
index = 4 * (4 - 1) / 2 + (3 - 1)  
index = 4 * 3 / 2 + 2  
index = 12 / 2 + 2  
index = 6 + 2  
index = 8
```

Step 3: Store value.

```
m->A[8] = 9;
```

So `9` is stored at `A[8]`.

## 25. Dry run of `set` for a structural zero

Call:

```
set(&m, 2, 4, 99);
```

Values:

```
i = 2  
j = 4  
x = 99
```

Step 1: Check condition.

```
2 >= 4
```

This is false.

So `(2,4)` is above the main diagonal.

Step 2: Do nothing.

The value `99` is ignored.

Why?

Because `M[2,4]` must be zero in a lower triangular matrix.

---

## 26. The `get` function

The `get` function returns the value at a matrix position.

Code:

```
int get(struct Matrix m, int i, int j) {  
    if (i >= j) {  
        int index = (i * (i - 1) / 2) + (j - 1);  
        return m.A[index];  
    }  
}
```

```
    return 0;  
}
```

### What the function does

If:

```
i >= j
```

then the value is stored.

So the function calculates the array index and returns:

```
m.A[index]
```

If:

```
i < j
```

then the value is a structural zero.

So the function returns:

```
0
```

---

## 27. Why `get` does not receive a pointer

The `get` function only reads the matrix.

It does not modify the matrix.

So it can receive the matrix as a normal struct variable:

```
struct Matrix m
```

This means `get` receives a copy of the struct.

That is okay for this lesson because it only needs to read values.

Compare:

```
void set(struct Matrix *m, int i, int j, int x)
```

`set` receives a pointer because it modifies the matrix.

But:

```
int get(struct Matrix m, int i, int j)
```

`get` receives a normal struct because it only reads.

---

## 28. Why `get` uses `m.A`

Inside `get`, `m` is not a pointer.

So we use the dot operator:

```
m.A
```

Correct:

```
return m.A[index];
```

Wrong:

```
return m->A[index];
```

Why?

Because `m->A` is only used when `m` is a pointer.

---

## 29. Dry run of `get` for a stored position

Suppose the array represents this matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9 10
```

Stored array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Call:

```
get(m, 4, 3);
```

Values:

```
i = 4
j = 3
```

Step 1: Check condition.

```
4 >= 3
```

This is true.

Step 2: Calculate index.

```
index = 4 * (4 - 1) / 2 + (3 - 1)
index = 4 * 3 / 2 + 2
index = 6 + 2
index = 8
```

Step 3: Return value.

```
return m.A[8];
```

Since:

```
A[8] = 9
```

The answer is:

```
9
```

---

### 30. Dry run of `get` for a structural zero

Call:

```
get(m, 2, 4);
```

Values:

```
i = 2  
j = 4
```

Step 1: Check condition.

```
2 >= 4
```

This is false.

So `(2,4)` is above the main diagonal.

Step 2: Return zero.

```
return 0;
```

The function does not read the array.

It already knows the value must be zero.

---

### 31. The `display` function

The `display` function prints the full matrix.

Code:

```
void display(struct Matrix m) {
    for (int i = 1; i <= m.n; i++) {
        for (int j = 1; j <= m.n; j++) {
            if (i >= j) {
                int index = (i * (i - 1) / 2) + (j - 1);
                printf("%d ", m.A[index]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}
```

## What display does

The stored array contains only useful values.

But the user wants to see the full square matrix.

So `display` uses nested loops:

```
for (int i = 1; i <= m.n; i++) {
    for (int j = 1; j <= m.n; j++) {
        // print each position
    }
}
```

For each position:

- If `i >= j`, print the stored value.
- If `i < j`, print `0`.

---

## 32. Why display takes nested loops

A `4 x 4` matrix has:

```
4 x 4 = 16 visible positions
```

Even though we store only `10` values, the display must print all `16` positions.

So it uses:

```
outer loop = rows
inner loop = columns
```

This is why display takes  $O(N^2)$  time.

---

### 33. Dry run of display

Assume:

```
n = 4
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The full matrix should be:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

#### Row 1

Positions:

```
(1,1), (1,2), (1,3), (1,4)
```

- (1,1):  $1 \geq 1$ , print  $A[0] = 1$
- (1,2):  $1 < 2$ , print 0
- (1,3):  $1 < 3$ , print 0
- (1,4):  $1 < 4$ , print 0

Output:

```
1 0 0 0
```

#### Row 2

- (2,1):  $2 \geq 1$ , print  $A[1] = 2$
- (2,2):  $2 \geq 2$ , print  $A[2] = 3$

- (2,3) : 2 < 3 , print 0
- (2,4) : 2 < 4 , print 0

Output:

2 3 0 0

### Row 3

- (3,1) : print A[3] = 4
- (3,2) : print A[4] = 5
- (3,3) : print A[5] = 6
- (3,4) : print 0

Output:

4 5 6 0

### Row 4

- (4,1) : print A[6] = 7
- (4,2) : print A[7] = 8
- (4,3) : print A[8] = 9
- (4,4) : print A[9] = 10

Output:

7 8 9 10

Full output:

1 0 0 0  
2 3 0 0  
4 5 6 0  
7 8 9 10

## 34. Full input reading strategy

The program asks the user to enter all  $N \times N$  matrix values.

For example, for  $n = 4$ , the user enters 16 values:

```
2 0 0 0
1 3 0 0
4 5 6 0
7 8 9 10
```

The program reads every value using nested loops:

```
for (int i = 1; i <= m.n; i++) {
    for (int j = 1; j <= m.n; j++) {
        scanf("%d", &x);
        set(&m, i, j, x);
    }
}
```

This means `set` is called for every matrix position.

But `set` only stores values where:

```
i >= j
```

So values above the diagonal are ignored automatically.

---

## 35. Why reading full input is useful

The user can type the matrix normally, as a full square matrix.

Example:

```
2 0 0 0
1 3 0 0
4 5 6 0
7 8 9 10
```

This is easy for the user to understand.

The program handles the storage internally.

The user does not need to know the 1D array order.

That is the job of the `set` function.

---

### 36. Dry run of full input for `n = 4`

For `n = 4`, storage size is:

$$4(4 + 1) / 2 = 10$$

So the array has 10 spaces:

`A[0]` to `A[9]`

Suppose the user enters:

```
2 0 0 0
1 3 0 0
4 5 6 0
7 8 9 10
```

#### Row 1 input

```
2 0 0 0
```

Calls:

```
set(&m, 1, 1, 2) -> stored
set(&m, 1, 2, 0) -> ignored
set(&m, 1, 3, 0) -> ignored
set(&m, 1, 4, 0) -> ignored
```

Only `2` is stored.

#### Row 2 input

```
1 3 0 0
```

Calls:

```
set(&m, 2, 1, 1) -> stored
set(&m, 2, 2, 3) -> stored
```

```
set(&m, 2, 3, 0) -> ignored  
set(&m, 2, 4, 0) -> ignored
```

Only **1** and **3** are stored.

### Row 3 input

```
4 5 6 0
```

Calls:

```
set(&m, 3, 1, 4) -> stored  
set(&m, 3, 2, 5) -> stored  
set(&m, 3, 3, 6) -> stored  
set(&m, 3, 4, 0) -> ignored
```

Only **4**, **5**, and **6** are stored.

### Row 4 input

```
7 8 9 10
```

Calls:

```
set(&m, 4, 1, 7) -> stored  
set(&m, 4, 2, 8) -> stored  
set(&m, 4, 3, 9) -> stored  
set(&m, 4, 4, 10) -> stored
```

All four values are stored.

Final stored array:

```
A = [2, 1, 3, 4, 5, 6, 7, 8, 9, 10]
```

## 37. Full C program: row-major lower triangular matrix

```
#include <stdio.h>
#include <stdlib.h>

struct Matrix {
    int *A;
    int n;
};

void set(struct Matrix *m, int i, int j, int x) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        m->A[index] = x;
    }
}

int get(struct Matrix m, int i, int j) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        return m.A[index];
    }
    return 0;
}

void display(struct Matrix m) {
    for (int i = 1; i <= m.n; i++) {
        for (int j = 1; j <= m.n; j++) {
            if (i >= j) {
                int index = (i * (i - 1) / 2) + (j - 1);
                printf("%d ", m.A[index]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

int main() {
    struct Matrix m;
    int x;

    printf("Enter dimension: ");
    scanf("%d", &m.n);
```

```

int totalElements = m.n * (m.n + 1) / 2;
m.A = (int *)malloc(totalElements * sizeof(int));

printf("Enter all matrix elements:\n");

for (int i = 1; i <= m.n; i++) {
    for (int j = 1; j <= m.n; j++) {
        scanf("%d", &x);
        set(&m, i, j, x);
    }
}

printf("\nMatrix is:\n");
display(m);

free(m.A);

return 0;
}

```

## 38. Explanation of the full program

### Header files

```

#include <stdio.h>
#include <stdlib.h>

```

`stdio.h` is used for:

```

printf()
scanf()

```

`stdlib.h` is used for:

```

malloc()
free()

```

### Struct definition

```

struct Matrix {
    int *A;

```

```
int n;  
};
```

This creates a matrix structure with:

- `A`: dynamic array for lower triangular values
- `n`: matrix dimension

## Set function

```
void set(struct Matrix *m, int i, int j, int x)
```

This stores `x` only when `i >= j`.

It uses a pointer because it changes the original matrix.

## Get function

```
int get(struct Matrix m, int i, int j)
```

This returns:

- stored value if `i >= j`
- `0` if `i < j`

## Display function

```
void display(struct Matrix m)
```

This prints the full square matrix.

It prints real stored values in the lower triangle and zeros above the diagonal.

## Main function

The `main` function:

1. Creates a matrix variable.
2. Reads the dimension.
3. Calculates how many values must be stored.
4. Allocates memory using `malloc()`.
5. Reads all matrix values.
6. Stores only lower triangular values using `set()`.

- 7. Displays the full matrix.
  - 8. Frees memory using `free()`.
- 

### 39. Important difference: `m->n` vs `m.n`

This is one of the most important C ideas in Lesson 7.

Use:

```
m->n
```

when `m` is a pointer.

Use:

```
m.n
```

when `m` is a normal struct variable.

---

### 40. Dot operator `.`

Use dot when you have a normal struct variable.

Example:

```
struct Matrix m;  
m.n = 4;
```

Here, `m` is the actual struct variable.

So we use:

```
m.n  
m.A
```

Example inside `get`:

```
int get(struct Matrix m, int i, int j) {  
    return m.A[index];  
}
```

Here, `m` is not a pointer.

So we use dot.

---

## 41. Arrow operator `->`

Use arrow when you have a pointer to a struct.

Example:

```
void set(struct Matrix *m, int i, int j, int x)
```

Here, `m` is a pointer.

So we use:

```
m->A  
m->n
```

Example:

```
m->A[index] = x;
```

This means:

```
Go to the struct pointed to by m, then access A.
```

---

## 42. Dot vs arrow summary

| Situation              | Operator           | Example                     |
|------------------------|--------------------|-----------------------------|
| Normal struct variable | <code>.</code>     | <code>m.A[index]</code>     |
| Pointer to struct      | <code>-&gt;</code> | <code>m-&gt;A[index]</code> |

Memory trick:

Dot means direct access.  
Arrow means follow the pointer, then access.

### 43. Why `set(&m, ...)` uses `&`

The `set` function is declared like this:

```
void set(struct Matrix *m, int i, int j, int x)
```

It expects a pointer.

So when we call it, we pass the address of `m`:

```
set(&m, i, j, x);
```

The `&` symbol means address.

So:

```
&m
```

means:

```
the address of m
```

This allows `set` to modify the original matrix.

### 44. Why `get(m, ...)` does not use `&`

The `get` function is declared like this:

```
int get(struct Matrix m, int i, int j)
```

It expects a normal struct value.

So we call it like this:

```
get(m, i, j);
```

not:

```
get(&m, i, j);
```

Also, `get` only reads the matrix.

It does not need to modify it.

---

## 45. Column-major update

Lesson 7 also explains how to convert the program from row-major storage to column-major storage.

The matrix type does not change.

It is still lower triangular.

The stored condition does not change:

```
i >= j
```

The zero condition does not change:

```
i < j
```

The number of stored values does not change:

```
n * (n + 1) / 2
```

Only the index formula changes.

---

## 46. Column-major formula

For lower triangular column-major storage, the formula is:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

This formula should be used only when:

```
i >= j
```

The formula means:

```
index = skipped columns + movement inside current column
```

---

## 47. Column-major formula inside `set`

Inside `set`, the matrix parameter is a pointer:

```
struct Matrix *m
```

So we must use:

```
m->n
```

Column-major index inside `set`:

```
int index = m->n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Column-major `set` function:

```
void set(struct Matrix *m, int i, int j, int x) {  
    if (i >= j) {  
        int index = m->n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);  
        m->A[index] = x;  
    }  
}
```

## 48. Column-major formula inside `get`

Inside `get`, the matrix parameter is not a pointer:

```
struct Matrix m
```

So we use:

```
m.n
```

Column-major index inside `get`:

```
int index = m.n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Column-major `get` function:

```
int get(struct Matrix m, int i, int j) {  
    if (i >= j) {  
        int index = m.n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);  
        return m.A[index];  
    }  
    return 0;  
}
```

---

## 49. Column-major formula inside `display`

Inside `display`, the matrix parameter is also not a pointer:

```
struct Matrix m
```

So we use:

```
m.n
```

Column-major `display` function:

```

void display(struct Matrix m) {
    for (int i = 1; i <= m.n; i++) {
        for (int j = 1; j <= m.n; j++) {
            if (i >= j) {
                int index = m.n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
                printf("%d ", m.A[index]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

## 50. What changes when switching to column-major?

Only the formula changes.

| Program part     | Row-major                         | Column-major                                         |
|------------------|-----------------------------------|------------------------------------------------------|
| Stored condition | <code>i &gt;= j</code>            | <code>i &gt;= j</code>                               |
| Zero condition   | <code>i &lt; j</code>             | <code>i &lt; j</code>                                |
| Array size       | <code>n(n + 1) / 2</code>         | <code>n(n + 1) / 2</code>                            |
| Storage order    | Row by row                        | Column by column                                     |
| Formula          | <code>i(i - 1)/2 + (j - 1)</code> | <code>n(j - 1) - ((j - 2)(j - 1)/2) + (i - j)</code> |

This is an important pattern in special matrices:

The structure can stay the same, but the formula can change.

## 51. Full column-major version of the important functions

```

void set(struct Matrix *m, int i, int j, int x) {
    if (i >= j) {
        int index = m->n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
        m->A[index] = x;
    }
}

```

```

int get(struct Matrix m, int i, int j) {
    if (i >= j) {
        int index = m.n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
        return m.A[index];
    }
    return 0;
}

void display(struct Matrix m) {
    for (int i = 1; i <= m.n; i++) {
        for (int j = 1; j <= m.n; j++) {
            if (i >= j) {
                int index = m.n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
                printf("%d ", m.A[index]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

## 52. Row-major vs column-major example

Matrix:

```

1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10

```

### Row-major array

Row-major stores row by row:

```

Row 1: 1
Row 2: 2 3
Row 3: 4 5 6
Row 4: 7 8 9 10

```

Array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

### Column-major array

Column-major stores column by column:

```
Column 1: 1 2 4 7  
Column 2: 3 5 8  
Column 3: 6 9  
Column 4: 10
```

Array:

```
A = [1, 2, 4, 7, 3, 5, 8, 6, 9, 10]
```

The visible matrix is the same.

Only the 1D array order changes.

---

## 53. Complexity of Lesson 7

### Storage complexity

A lower triangular matrix stores:

```

$$N(N + 1) / 2$$

```

This is about half of full matrix storage.

In Big-O notation, this is still:

```

$$O(N^2)$$

```

Why?

Because:

```

$$N(N + 1) / 2$$

```

still grows like  $N^2$  when  $N$  becomes large.

So lower triangular storage is:

$O(N^2)$ , but optimized to about half of full storage

### `set()` complexity

The `set` function does:

- one condition check
- one formula calculation if needed
- one array assignment if needed

It does not use loops.

So:

`set()` =  $O(1)$

### `get()` complexity

The `get` function does:

- one condition check
- one formula calculation if needed
- one array lookup if needed

It does not use loops.

So:

`get()` =  $O(1)$

## Input reading complexity

The program reads all  $N \times N$  values from the user.

That means it reads:

$N^2$  values

So:

```
input reading =  $O(N^2)$ 
```

Even though only half the values are stored, the program still reads the full matrix.

### Display complexity

The display function prints all  $N \times N$  visible positions.

So:

```
display() =  $O(N^2)$ 
```

## 54. Complexity table

| Operation              | Time complexity | Space complexity   | Reason                                 |
|------------------------|-----------------|--------------------|----------------------------------------|
| Matrix storage         | —               | $O(N^2)$ optimized | Stores $N(N + 1) / 2$ values.          |
| <code>set()</code>     | $O(1)$          | —                  | One condition and one formula.         |
| <code>get()</code>     | $O(1)$          | —                  | One condition and one possible lookup. |
| Input reading          | $O(N^2)$        | —                  | Reads every visible matrix position.   |
| <code>display()</code> | $O(N^2)$        | —                  | Prints every visible matrix position.  |

Important correction:

```
Lower triangular storage is not  $O(N)$ .  
It is  $O(N^2)$ , but it stores about half of a full  $N \times N$  matrix.
```

## 55. Common beginner mistakes

### Mistake 1: Using the diagonal condition

Wrong:

```
if (i == j)
```

Correct for lower triangular matrix:

```
if (i >= j)
```

Reason:

A lower triangular matrix stores the main diagonal and everything below it.

---

## Mistake 2: Storing values above the diagonal

Wrong idea:

M[2,4] should be stored.

Correct idea:

M[2,4] is above the diagonal because  $2 < 4$ .  
So it is a structural zero.

---

## Mistake 3: Forgetting `j - 1` in row-major formula

Wrong:

```
index = i * (i - 1) / 2 + j;
```

Correct:

```
index = i * (i - 1) / 2 + (j - 1);
```

Reason:

Matrix positions start at `1`, but array indexes start at `0`.

---

#### Mistake 4: Using `m.A` inside `set`

Wrong:

```
void set(struct Matrix *m, int i, int j, int x) {  
    m.A[index] = x;  
}
```

Correct:

```
void set(struct Matrix *m, int i, int j, int x) {  
    m->A[index] = x;  
}
```

Reason:

Inside `set`, `m` is a pointer.

Pointers use `->`.

---

#### Mistake 5: Using `m->A` inside `get`

Wrong:

```
int get(struct Matrix m, int i, int j) {  
    return m->A[index];  
}
```

Correct:

```
int get(struct Matrix m, int i, int j) {  
    return m.A[index];  
}
```

Reason:

Inside `get`, `m` is a normal struct variable.

Normal struct variables use `.`.

---

### Mistake 6: Forgetting `&m` when calling `set`

Wrong:

```
set(m, i, j, x);
```

Correct:

```
set(&m, i, j, x);
```

Reason:

`set` expects a pointer to the matrix.

So we pass the address of `m`.

---

### Mistake 7: Using `&m` when calling `get`

Wrong for this lesson's version:

```
get(&m, i, j);
```

Correct:

```
get(m, i, j);
```

Reason:

`get` expects a normal struct variable, not a pointer.

---

### Mistake 8: Forgetting to allocate enough memory

Wrong:

```
m.A = (int *)malloc(m.n * sizeof(int));
```

This only allocates `n` values, which is enough for a diagonal matrix, not a lower triangular matrix.

Correct:

```
m.A = (int *)malloc((m.n * (m.n + 1) / 2) * sizeof(int));
```

Reason:

A lower triangular matrix stores  $n(n + 1) / 2$  values.

---

### Mistake 9: Forgetting to free memory

Wrong:

```
// program ends without free(m.A)
```

Correct:

```
free(m.A);
```

Reason:

Memory allocated using `malloc()` should be released using `free()`.

---

### Mistake 10: Thinking lower triangular storage is $O(N)$

Wrong idea:

```
Lower triangular matrix storage is  $O(N)$ .
```

Correct idea:

```
Lower triangular matrix storage is  $O(N^2)$ , but about half of full matrix storage.
```

Reason:

It stores:

```
 $N(N + 1) / 2$ 
```

This still grows like  $N^2$ .

---

### Mistake 11: Thinking row-major and column-major display different matrices

Wrong idea:

Row-major and column-major create different matrix shapes.

Correct idea:

They store the same matrix shape, but the 1D array order is different.

The visible matrix should be the same if the correct formula is used.

---

## 56. Beginner mental model

Think of the full matrix as what the user sees.

Example:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9 10
```

Think of the 1D array as what the computer stores.

For row-major storage:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The program works like this:

1. If the position is in the lower triangle, calculate the array index.
  2. If the position is above the diagonal, return or print `0`.
  3. The zeros above the diagonal are not stored.
  4. The formula connects the visible matrix position to the hidden array position.
-

## 57. Main thinking pattern for Lesson 7

For any lower triangular matrix position  $(i, j)$ , ask:

### Question 1

Is the position stored?

Check:

```
i >= j
```

If true, it is stored.

If false, it is a structural zero.

### Question 2

If stored, what is the array index?

For row-major:

```
index = i * (i - 1) / 2 + (j - 1)
```

For column-major:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

### Question 3

Which operator should I use?

If the function receives:

```
struct Matrix *m
```

use:

```
m->A  
m->n
```

If the function receives:

```
struct Matrix m
```

use:

```
m.A  
m.n
```

---

## 58. Practice example 1

Given:

```
n = 4
```

Find the storage size.

Formula:

```
n(n + 1) / 2
```

Substitute:

```
4(4 + 1) / 2 = 4(5) / 2 = 10
```

Answer:

```
The array needs 10 cells.
```

---

## 59. Practice example 2

Is `M[3,2]` stored?

Check:

$3 \geq 2$

This is true.

Answer:

Yes,  $M[3,2]$  is stored.

---

### 60. Practice example 3

Is  $M[2,4]$  stored?

Check:

$2 \geq 4$

This is false.

Answer:

No,  $M[2,4]$  is a structural zero.

---

### 61. Practice example 4

Find row-major index of  $M[4,3]$ .

Values:

$i = 4$   
 $j = 3$

Check:

$4 \geq 3$

True.

Formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

Substitute:

```
index = 4 * (4 - 1) / 2 + (3 - 1)
index = 4 * 3 / 2 + 2
index = 6 + 2
index = 8
```

Answer:

```
M[4,3] -> A[8]
```

---

## 62. Practice example 5

Find row-major index of `M[5,2]` .

Values:

```
i = 5
j = 2
```

Check:

```
5 >= 2
```

True.

Formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

Substitute:

```
index = 5 * (5 - 1) / 2 + (2 - 1)
index = 5 * 4 / 2 + 1
index = 10 + 1
index = 11
```

Answer:

```
M[5,2] -> A[11]
```

---

## 63. Practice example 6

What does `get(m, 2, 5)` return?

Check:

```
2 >= 5
```

False.

So `(2,5)` is above the diagonal.

Answer:

```
get(m, 2, 5) returns 0.
```

Reason:

```
M[2,5] is a structural zero.
```

---

## 64. Practice example 7

Inside this function:

```
void set(struct Matrix *m, int i, int j, int x)
```

Should we use `m.A` or `m->A`?

Answer:

`m->A`

Reason:

`m` is a pointer.

---

## 65. Practice example 8

Inside this function:

```
int get(struct Matrix m, int i, int j)
```

Should we use `m.A` or `m->A`?

Answer:

`m.A`

Reason:

`m` is a normal struct variable.

---

## 66. Key formulas from Lesson 7

**Stored condition**

`i >= j`

**Zero condition**

`i < j`

### Stored values

```
n * (n + 1) / 2
```

### Structural zeros

```
n * (n - 1) / 2
```

### Row-major index formula

```
index = i * (i - 1) / 2 + (j - 1)
```

### Column-major index formula

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

### Dynamic allocation

```
m.A = (int *)malloc((m.n * (m.n + 1) / 2) * sizeof(int));
```

### Free memory

```
free(m.A);
```

---

## 67. Important code snippets

### Struct

```
struct Matrix {  
    int *A;  
    int n;  
};
```

### Row-major set

```
void set(struct Matrix *m, int i, int j, int x) {  
    if (i >= j) {
```

```

        int index = (i * (i - 1) / 2) + (j - 1);
        m->A[index] = x;
    }
}

```

### Row-major `get`

```

int get(struct Matrix m, int i, int j) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        return m.A[index];
    }
    return 0;
}

```

### Row-major `display`

```

void display(struct Matrix m) {
    for (int i = 1; i <= m.n; i++) {
        for (int j = 1; j <= m.n; j++) {
            if (i >= j) {
                int index = (i * (i - 1) / 2) + (j - 1);
                printf("%d ", m.A[index]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

### Full input loop

```

for (int i = 1; i <= m.n; i++) {
    for (int j = 1; j <= m.n; j++) {
        scanf("%d", &x);
        set(&m, i, j, x);
    }
}

```

## 68. Lesson 7 final summary

Lesson 7 teaches how to implement a lower triangular matrix in C.

A lower triangular matrix stores positions where:

$i \geq j$

Positions where:

$i < j$

are structural zeros.

The matrix stores only:

$n * (n + 1) / 2$

values instead of  $n * n$  values.

The matrix is represented using:

```
struct Matrix {  
    int *A;  
    int n;  
};
```

The 1D array is created using:

`malloc()`

and released using:

`free()`

The row-major formula is:

```
index = i * (i - 1) / 2 + (j - 1)
```

The column-major formula is:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)
```

The `set` function stores values only in the lower triangle.

The `get` function returns stored values from the lower triangle and returns `0` for structural zeros.

The `display` function prints the full matrix using stored values and fake zeros.

The most important C detail is:

```
Use m->A and m->n when m is a pointer.  
Use m.A and m.n when m is a normal struct variable.
```

The biggest idea of Lesson 7 is:

```
The matrix looks like a full square matrix to the user,  
but the computer stores only the useful lower triangular part.
```

That is the heart of special matrix storage in C.